

CPMS04

Britt Anderson

Winter 2025

1 Neural Networks

Our goal here is to understand the basic mathematical objects and their notation that underlie neural networks. We want to have enough of a grasp to be able to code simple versions of them by hand. If you ever build a big model you will undoubtedly want to use the well established standard libraries, but for making design decisions, thinking about aberrant behavior, or even just conceiving the kind of network you want to build and how it will match your problem, you want a more fundamental and practical grasp of the pieces you will be putting together.

Our distal goal is to code the Hopfield Network, and ideally the Kohonen self-organizing map, but along the way we will first learn about vectors, matrices, scalar products, the perceptron, and learning rules.

1.1 John Hopfield

To motivate our learning, and to put a human face on the project these two links give you a glimpse of Hopfield talking about his work at the 2024 Nobel Prize lectures, and a nice long podcast that gives a slow, thorough, tutorial discussion of what it is all about. We will come back to this later, but for now start listening. There is an associated homework.

1.1.1 Nobel Prize

John Hopfield's Nobel Lecture

1.1.2 Podcast on the Hopfield Network

This theoretical neuroscience podcast gives a nice long look at the Hopfield Network. It works up to its creation, its abstraction, and its social impact on

shaping the field of computational neuroscience. It is long, but, if you make your way through it you will have a good overview of the model before you start coding it.

1.2 The Math That Underlies Neural Networks?

1.2.1 Linear Algebra

The math at the heart of neural networks and their computer implementation is **linear algebra**. For us, the section of linear algebra we are going to need is mostly limited to vectors, matrices and how to add and multiply them.

Class Discussion

What makes linear algebra linear?

1. Important Objects and Operations

- Vectors
- Matrices
- Scalars
- Addition
- Multiplication (scalar and matrix)
- Transposition
- Inverse

Further Class Discussion

Give a definition or description of each of these objects and operations.

1.2.2 Adding Matrices

To gain some hands on familiarity with the manipulation of matrices and vectors we will try to do some hand and programming exercises for some of the fundamental operations of addition and multiplication. We will also thereby learn that some of the rules we learned for numbers (such as $a * b = b * a$) do not always apply in other mathematical realms.

There are in fact many ways to think about what a vector is.

It can be thought of as a column (or row of numbers). More abstractly it is an object (arrow) with magnitude and direction. Most abstractly it is anything that obeys the requirements of a vector space.

For particular circumstances one or another of the different definitions may serve our purposes better. In application to neural networks we often just use the first definition, a column of numbers, but the second can be more helpful for developing our geometric intuitions about what various learning rules are doing and how they do it.

Similarly, we often just consider a matrix as a collection of vectors or as a rectangular (2-D) collection of numbers.

1.2.3 Activity

In your programming language of choice find the package (sometimes also called a library) that has the construct of a "vector". Is it the same thing as the vector described above? If not, make sure you find the equivalent construct for your programming language. Make sure you can use it to create two vectors and compute their scalar product. Be prepared to demonstrate this for the class.

```
(require math/matrix)
(define a-mat (matrix [[1 2 3]]))
(define b-mat (matrix [[4 5 6]]))
;adding two vectors
(matrix+ a-mat b-mat)
(matrix* a-mat (matrix-transpose b-mat))
```

Figure 1: This demonstrates the idea for the racket programming language where the library for dealing with vector like things is called `math/matrix`.

IAMHERE

- 1.2.4 Common Notational Conventions for Vectors and Matrices}
- 1.2.5 Non-linearities}
- 1.2.6 Exercise XOR}
- 1.2.7 Connections}
- 1.2.8 Boolean Logic}
- 1.2.9 First Order Logic - Truth Tables}
- 1.2.10 How does a network like this work?}
- 1.2.11 Test your understanding:}
- 1.2.12 A Worked Example}
- 1.2.13 Distance Metrics}
- 1.2.14 Hebb's @[hyperlink\["https://en.wikipedia.org/wiki/Outer_](https://en.wikipedia.org/wiki/Outer_product)
product"]{Outer Product} Rule}